

File Permissions in Linux

Portfolio Activity: Managing Authorization with Linux Commands

Role	Security professional supporting a research team
Scenario focus	Review and correct file and directory permissions in /home/researcher2/projects
Primary tools	ls -la and chmod
Security objective	Authorize appropriate access while removing unnecessary or unauthorized permissions

Project Description

This activity documents how Linux commands can be used to examine and manage authorization through file and directory permissions. In the scenario, a security professional reviews permissions in the projects directory for a research team, identifies access that does not match the organization's authorization requirements, and updates permissions to reduce unauthorized access.

The main goal is to verify who can read, write, and execute files or directories, then use chmod to correct any permissions that are too open. This supports the principle of least privilege by giving users only the access they need to perform their role.

1. Check File and Directory Details

To review the permissions for all files and subdirectories, including hidden files, I would first move into the projects directory and run the long listing command with the all option.

```
$ cd /home/researcher2/projects
$ ls -la
```

The ls -la command lists files and directories in long format and includes hidden items. The long format output displays the 10-character permissions string, owner, group, file size, modification date, and file or directory name.

Item	Current permission string	Permission meaning	Assessment
project_k.txt	-rw-rw-rw-	User: read/write; Group: read/write; Other: read/write	Other has write access and must be corrected.
project_m.txt	-rw-r-----	User: read/write; Group: read; Other: none	Permissions are appropriate.
project_r.txt	-rw-rw-r--	User: read/write; Group: read/write; Other: read	Permissions are acceptable because other does not have write access.

project_t.txt	-rw-rw-r--	User: read/write; Group: read/write; Other: read	Permissions are acceptable because other does not have write access.
.project_x.txt	-rw--w----	User: read/write; Group: write; Other: none	Hidden file permissions must be changed so user and group can read, and no one can write.
drafts	drwx--x---	User: read/write/execute; Group: execute; Other: none	Directory permissions must be changed so only researcher2 can access it.

2. Describe the Permissions String

A Linux permissions string has 10 characters. The first character identifies the item type, such as a dash (-) for a regular file or d for a directory. The next nine characters are grouped into three sets of three: user permissions, group permissions, and other permissions.

Example: -rw-rw-r-- project_t.txt

Character 1: - = regular file

Characters 2-4: rw- = user can read and write, but cannot execute

Characters 5-7: rw- = group can read and write, but cannot execute

Characters 8-10: r-- = other users can read only

In this example, project_t.txt is a regular file. The owner and group can read and write the file, while other users can only read it. This means other users cannot modify or delete the file content.

3. Change File Permissions

The organization does not allow the other permission category to have write access to any files. Based on the current permissions, project_k.txt is the file that needs to be corrected because other users have read and write access.

```
$ chmod o-w project_k.txt
$ ls -l project_k.txt
-rw-rw-r-- 1 researcher2 research_team 46 Mar 20 14:30 project_k.txt
```

The chmod o-w project_k.txt command removes write permission from other users. The updated permission string is -rw-rw-r--, which means the owner and group can read and write, but other users can only read. This removes unauthorized write access while still allowing read access where permitted.

4. Change File Permissions on a Hidden File

The hidden file `.project_x.txt` is archived and should not have write access for anyone. However, the user and group should be able to read the file. Because it is hidden, the `ls -la` command is needed to display it.

```
$ chmod u-w,g-w,g+r .project_x.txt
$ ls -la .project_x.txt
-r--r----- 1 researcher2 research_team 46 Mar 20 14:35 .project_x.txt
```

The command removes write permission from the user and group, then adds read permission for the group. The final permission string, `-r--r-----`, means the user and group can read the file, but no category has write or execute permission. Other users have no access.

5. Change Directory Permissions

The drafts directory belongs to the researcher2 user, and only researcher2 should be allowed to access the directory and its contents. The current permission string gives the group execute permission, which may allow group members to enter the directory if they know the file names inside it. This access should be removed.

```
$ chmod g-x drafts
$ ls -ld drafts
drwx----- 2 researcher2 research_team 4096 Mar 20 14:40 drafts
```

The `chmod g-x drafts` command removes execute permission from the group. The final permission string, `drwx-----`, means researcher2 can read, write, and enter the directory, while the group and other users have no access. This matches the authorization requirement for the drafts directory.

Final Permission State

Item	Final permission string	Result
project_k.txt	-rW-rW-r--	Other write permission removed.
project_m.txt	-rW-r-----	No change needed.
project_r.txt	-rW-rW-r--	No change needed.
project_t.txt	-rW-rW-r--	No change needed.
.project_x.txt	-r--r-----	User and group can read; no one can write.
drafts	drwx-----	Only researcher2 can access the directory.

Summary

In this activity, I used Linux commands to inspect and modify file permissions for a research team's projects directory. I used `ls -la` to view all files, including the hidden file `.project_x.txt`, and interpreted the 10-character permissions strings to determine whether the current permissions matched the organization's authorization needs.

I then used `chmod` to remove unauthorized write access from `project_k.txt`, update the archived hidden file so only the user and group could read it, and restrict the drafts directory so only `researcher2` could access it. These changes strengthen authorization controls and help enforce least privilege across files and directories.

Portfolio Reflection

This activity demonstrates practical Linux administration skills that are useful in cybersecurity roles. File permissions directly affect confidentiality and integrity because improper permissions can expose sensitive files or allow unauthorized users to modify data.

By checking permissions, identifying misconfigurations, and applying the correct `chmod` changes, I showed how command-line tools can be used to support secure access control in a real operational environment.

Activity Requirements Checklist

Requirement	Status
Typed versions of the Linux commands are included.	Completed
Each command includes a clear explanation.	Completed
Project description is included at the beginning.	Completed
Summary is included at the end.	Completed
<code>chmod</code> is used to update file and directory permissions.	Completed
<code>ls -la</code> is used to check file permissions and hidden files.	Completed
The 10-character permissions string is explained.	Completed
Hidden files and directories are discussed.	Completed

Source materials used: Current file permissions and Instructions for including Linux commands documents provided with the activity.